

THANK YOU to Our Sponsors

PLATINUM



GOLD



SILVER





BEER CITY CODE 2026 · GRAND RAPIDS, MI · AUG 15

When Logs Tell the Whole Story

OpenTelemetry for Beginners

Barret Blake · Architect, NimblePros

#OTel | #Observability | #OpenTelemetry



WHO AM I

Barret Blake



Barret Blake

Architect · NimblePros



A few things about me

-  Microsoft MVP — .NET
-  Husband & Father
-  Coder, Blogger & Speaker
-  Mentor
-  Gamer
-  Model Railroader & LEGO Enthusiast
-  Buckeyes & Browns Fan

THE PROBLEM

Production failures are inevitable

What happens next is up to you.



No observability

Guessing in the dark. Each incident is a mystery. MTTR spirals.



Scattered logs

Logs exist but tell disconnected stories. Hard to correlate across services.



Structured telemetry

A clear, connected story. Logs trace the path from failure to root cause.

The difference between a quick fix and a prolonged outage **often comes down to whether your logs tell a clear, connected story.**

What is OpenTelemetry?

An open-source, language-agnostic, vendor-neutral framework for application observability — providing APIs, SDKs, and tools to capture traces, metrics, and logs.



Open Source

Backed by the CNCF. Community driven and widely adopted.



Language Agnostic

.NET, Java, Go, JS, Python, Rust, Ruby & more.



Vendor Neutral

Send your data to ANY compatible observability platform.

The industry standard for cloud-native observability.

The Three Signals of OTel



Traces

The full path of a request through your system — entry point to database and back.



Metrics

Numeric measurements over time — request counts, error rates, latency percentiles.



Logs

Timestamped records of what happened and why. The base unit of all telemetry.



Baggage — not a signal, but cross-cutting context: key-value data propagated across all spans and service boundaries within a trace.



Coming soon: Profiles — an emerging fourth signal (public alpha, 2026) answering “which code is burning CPU / memory?”

Traces: Following the Request

A trace records the complete path of a request through your application.

- ✓ Each unit of work = a Span
- ✓ Spans nest into parent / child trees
- ✓ Captures timing for every step
- ✓ Spans across services share a TraceId
- ✓ Critical for cross-service debugging

TRACE WATERFALL VIEW

mission.launch.initiated



POST /api/missions/launch



INSERT INTO launches



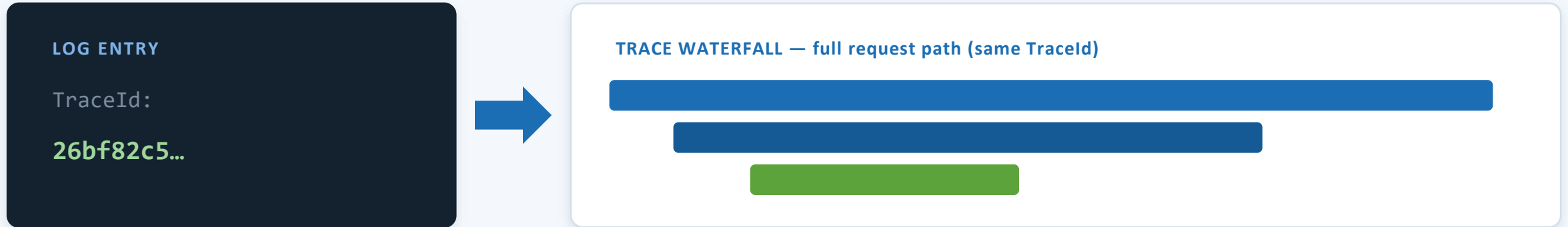
← Time →



Real-world impact: Analyzing trace waterfalls on a client project revealed several performance bottlenecks — a **10–40% runtime reduction** across key workflows.

Trace Context: Connecting the Dots

The same TraceId and SpanId appear in both your logs AND your trace spans.



One click to context

Jump from a log line directly to the full trace waterfall in your platform.



Cross-service correlation

The same TraceId flows through all services — find a user's request across microservices instantly.



No manual correlation

OTel propagates trace context automatically via HTTP headers and messaging metadata.

Metrics: Measuring What Matters



Error-Rate Alerting

Don't alert on every failure — you'll train your team to ignore alerts. Use a metric counter and fire only when a threshold is crossed (e.g. 20+ failures in 60 seconds).



Code Usage Analysis

Before investing in a refactor, use metrics to prove it's worth it. If a 1,000-line class runs hundreds of times per minute, that data is your business case.

```
// Custom counter setup
var meter = new Meter(
    "MissionControl.Telemetry");
var counter = meter
    .CreateCounter<long>(
        "missions_launched");

// In your endpoint:
counter.Add(1,
    new KeyValuePair<string, object>(
        "mission.name", missionName));
```



Metric output: 3 launches for Enterprise, 1 for Voyager.

Logs: The Story Behind the Numbers

“Not all logs are events, but all events are logs.” — OpenTelemetry docs



Automatically shipped

OTel exports the log records you already create — the exporter handles it, no special wiring.



Structured output

OTel turns any log format into a structured record: timestamp, severity, body, attributes.



Correlated with traces

Every log within a trace carries the `TraceId` and `SpanId` — connecting logs to the request.

When you need to understand what went wrong and why — logs are where you look.

Anatomy of an OTEL Log Record

```
LogRecord.Timestamp: 2026-08-15T09:14...
LogRecord.TraceId: 26bf82c5...
LogRecord.SpanId: c0da8620...
LogRecord.TraceFlags: Recorded
LogRecord.CategoryName: Program
LogRecord.Severity: Info
LogRecord.Body: Mission launch
    requested: {missionName}...
LogRecord.Attributes:
    missionName: Enterprise
    status: Launched
```

Field	Purpose
Timestamp	When the event occurred
TraceId	Links log to its trace
SpanId	Specific span within the trace
TraceFlags	W3C standard trace flags
CategoryName	.NET ILogger category
Severity	Log level (Info, Warn, Error...)
Body	The log message
Attributes	Structured key-value data

```
logger.LogInformation("Mission launch requested: {missionName}, status: {status}", missionName, status);
```

Baggage: Context That Travels

Key-value context that propagates automatically across all spans, services, and HTTP headers within a trace.

```
// Set baggage at request start
Baggage.SetBaggage(
    "mission.commander", "Picard");
Baggage.SetBaggage(
    "mission.priority", "High");

// Read anywhere in the trace
var commander = Baggage
    .GetBaggage("mission.commander");
```



Good uses for Baggage

- User ID for correlation
- Tenant ID in multi-tenant apps
- Source IP address
- Feature-flag or A/B testing group
- Request source / channel



Security warning: Baggage is added to HTTP headers and is readable by any receiving service. **Never put PII, passwords, or sensitive data in baggage.**

Choosing the Right Log Level

Level	Use When...	Recommended For
Trace	Highly detailed debugging. Very high volume. Dev only.	Never in production
Debug	Diagnostic details for development and debugging.	Dev / staging only
Information	Normal operations. Application milestones, key events.	Production — selectively
Warning	Unexpected but recoverable. Worth monitoring.	Always in production
Error	Operation failed. Needs attention; app still running.	Always in production
Critical	Catastrophic failure. Immediate action required.	Always + alerting

Common Logging Mistakes to Avoid



Over-Logging

Problem: Logging everything buries the signal in noise — and drives up cost on commercial platforms.

Fix: Log meaningful events at appropriate levels. Filter dev-only logs from production.



Unstructured Messages

Problem: String concatenation (“Error for user ” + id) can't be queried, filtered, or aggregated.

Fix: Use structured logging: `LogError(“User {UserId} failed checkout”, id)` — queryable attributes.



Missing Context

Problem: “Operation failed” is useless without which operation, which user, and what data.

Fix: Always include identifiers: user IDs, record IDs, operation names. Failures are data-specific.



Alert-Fatigue Loops

Problem: Alerting on every failure trains your team to build rules that ignore alerts entirely.

Fix: Use metric thresholds, not per-event alerts. Alert on patterns, not occurrences.

Setting Up OTel in .NET

Step 1 — Add NuGet Packages

-  OpenTelemetry.Extensions.Hosting
-  OpenTelemetry.Instrumentation.AspNetCore
-  OpenTelemetry.Exporter.OpenTelemetryProtocol

That's it for basic setup!

OTel auto-instruments your ASP.NET Core routes and starts collecting logs, traces, and metrics immediately. For Aspire projects it's included in the ServiceDefaults template — no manual setup.

```
// Step 2 – configure in Program.cs
builder.Logging.AddOpenTelemetry(o => o
    .SetResourceBuilder(ResourceBuilder
        .CreateDefault()
        .AddService("app-logger"))
    .AddOtlpExporter());

builder.Services.AddOpenTelemetry()
    .ConfigureResource(r => r
        .AddService("app-logger"))
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation()
        .AddOtlpExporter())
    .WithMetrics(m => m.AddOtlpExporter());
```

Exporting to Production Platforms



Self-Hosted

Jaeger — Distributed tracing

Grafana + Loki + Prometheus — Metrics, logs & dashboards

Elastic Stack — Search-first observability

Pros: control costs, data ownership. **Cons:** you own maintenance.



Commercial SaaS

Datadog — Full-stack observability

Grafana Cloud — Hosted Grafana + alerting

Azure App Insights — Native .NET + Azure

New Relic — AI-powered incident response

Pros: managed, enterprise features. **Cons:** cost scales with data.



The OTel advantage: portability

Everything speaks **OTLP**, the OpenTelemetry wire protocol. Route through the **OpenTelemetry Collector** and switching platforms is a config change — not a rewrite. Your data model stays the same.

Aspire: OTel Built In for Local Dev



Console View

Raw log output across all services in one pane.



Structured Logs

Filter and search by severity, category, or attribute.



Trace List

Browse all traces, filter by service or duration.



Waterfall View

Visual span timing — pinpoint slow code instantly.



Metrics

Runtime metrics and custom counters in one dashboard.



Multi-Language

Aspire 13 is now a multi-language platform — .NET, Python & JS/Node.



Don't deploy the Aspire dashboard to production. It exposes full debug logs with no auth. Use a dedicated observability platform in deployed environments.

Reusable Telemetry with ServiceDefaults

The ServiceDefaults shared project is the perfect home for reusable telemetry across your whole solution.



Custom Metric Counters

Define counters in ServiceDefaults via an AddMeter extension, then call counter.Add(1) in any endpoint. Metrics appear instantly in the dashboard.



Reusable Span Factory

A generic StartDatabaseSpan<T>() auto-tags span attributes from your entity's properties. One method, consistent tracing everywhere.

```
// ServiceDefaults: reusable span factory
public static Activity? StartDatabaseSpan<T>(
    this ActivitySource src, string name, T entity)
{
    var activity = src.StartActivity(
        name, ActivityKind.Internal);
    if (activity is null) return activity;
    // auto-tag entity properties...
    foreach (var p in typeof(T).GetProperties())
        activity.SetTag($"db.entity.{p.Name}",
                        p.GetValue(entity));

    return activity;
}
```



DEMO TIME

Mission Control

Launch a mission → watch traces, metrics, logs & baggage light up the Aspire dashboard in real time.

Traces

Metrics

Logs

Baggage

Mission Control — the app

Mission Control

Open the Aspire dashboard →

OTEL SIGNALS BEING EMITTED:

Traces

Metrics

Logs

Baggage

MISSION ROSTER

↻ Refresh

↺ Reset All

⚡ Generate Load

Enterprise NCC-1701

The Final Frontier · crew 430

DOCKED

Voyager NCC-74656

Delta Quadrant · crew 141

DOCKED

Defiant NX-74205

The Gamma Quadrant · crew 50

DOCKED

Enterprise-D NCC-1701-D

Sector 001 & Beyond · crew 1014

DOCKED

Titan NCC-80102

Deep-Space Survey · crew 350

DOCKED

LAUNCH CONTROL

Commander (populates W3C Baggage)

Kirk

Priority (populates W3C Baggage)

Routine

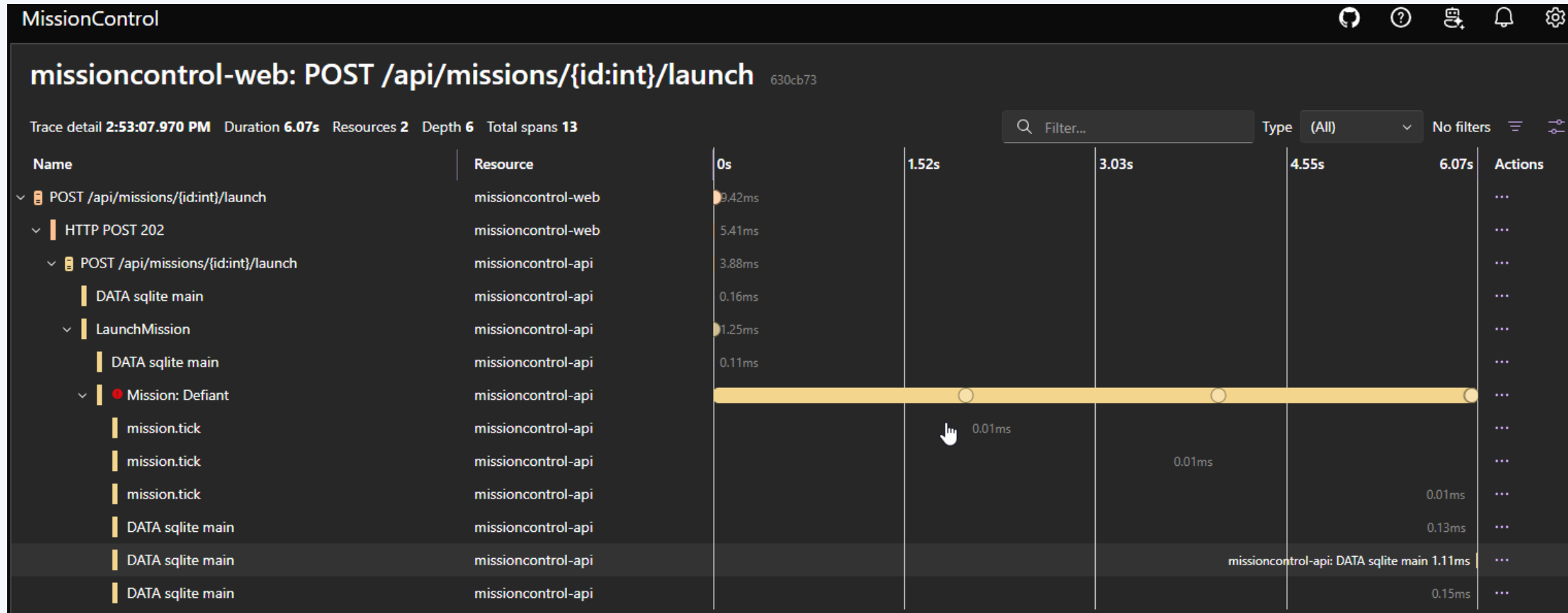
☐ Force failure (demo error span + error log)

Select a ship to launch

No mission selected.

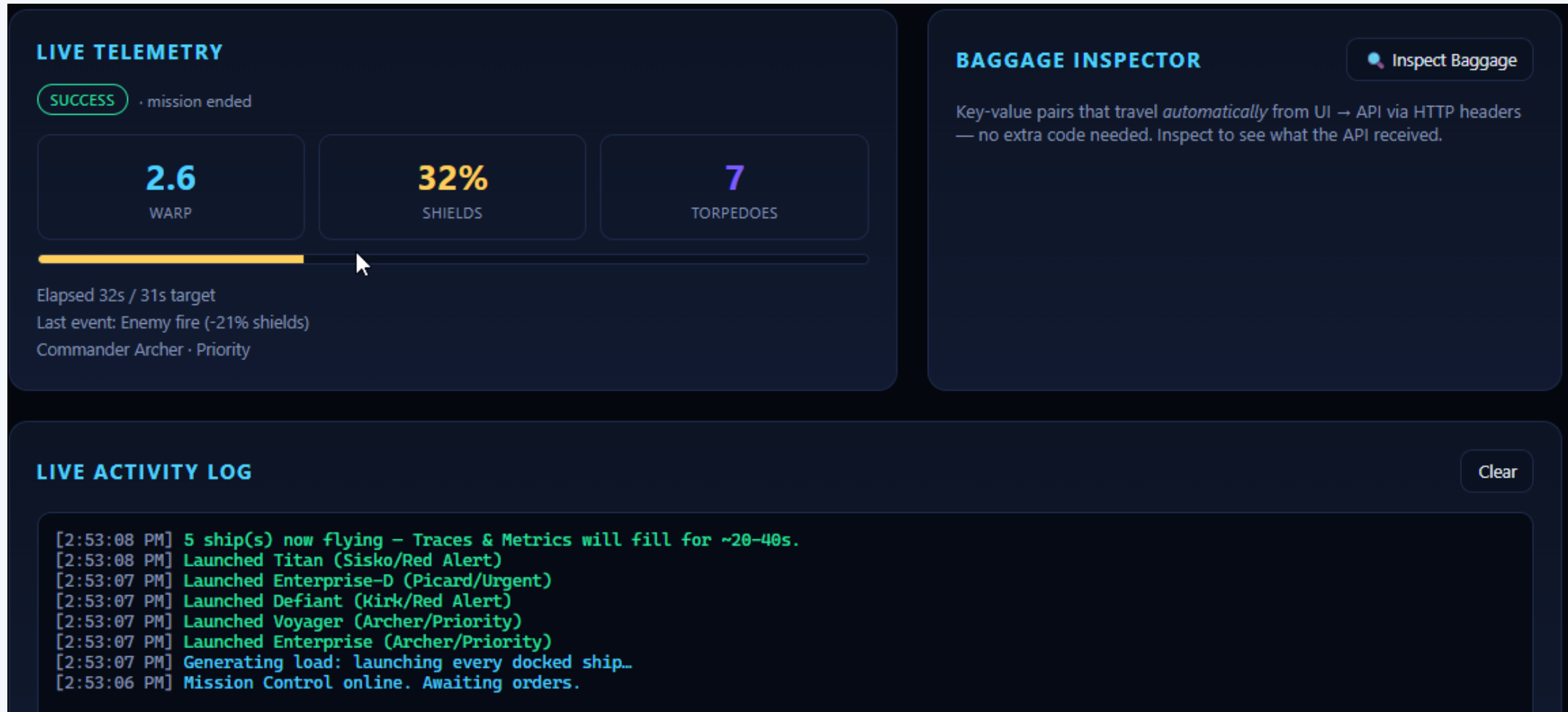
Every launch flows Web → API → database as one trace. Open the **Aspire dashboard** to see the **trace**, the **missions launched** metric, and the **structured logs** (each carries the **TraceId/SpanId**). Commander & priority ride along as **W3C Baggage**.

Aspire Dashboard — Trace Waterfall



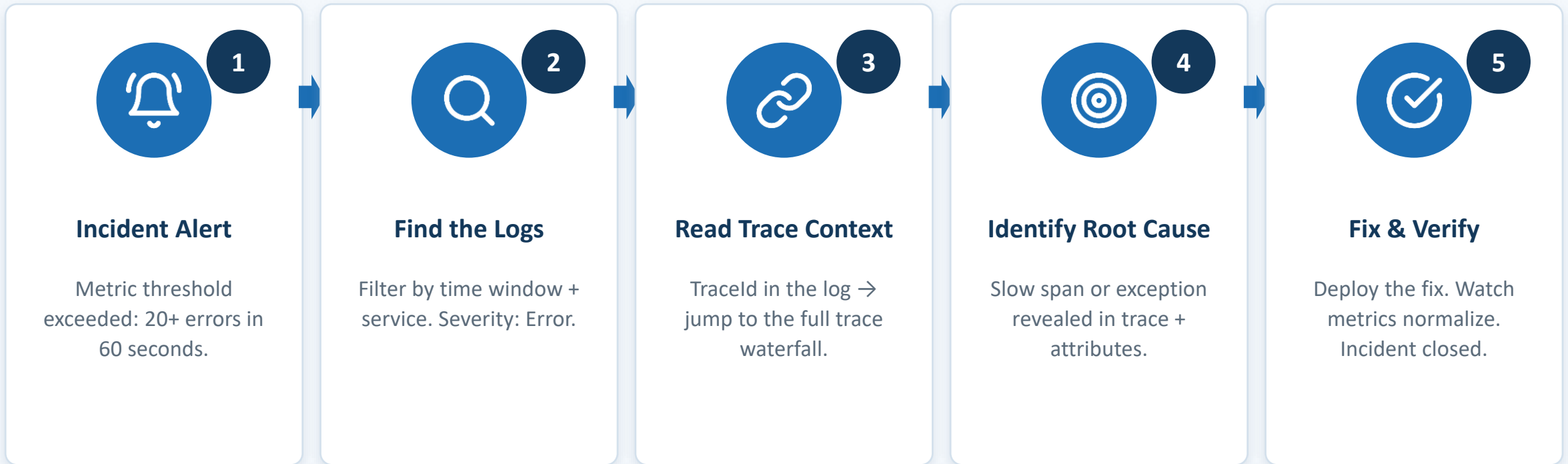
One long-lived Mission span; a **mission.tick** child every 2s captures warp, shields & torpedoes. Commander rides along as Baggage.

Aspire Dashboard — Metrics & Structured Logs



The same Traceld runs through every 2s tick — tying the gauges, the logs, and the Mission trace together.

The Observability Stack in Action



Key Takeaways



OpenTelemetry is the industry standard — vendor-neutral, language-agnostic observability for modern apps.



Avoid over-logging and alert fatigue. Alert on patterns with metric thresholds, not individual events.



Logs + Traces + Metrics work together. The Tracelid connects log entries to full request paths across services.



Aspire ServiceDefaults makes telemetry reusable. Define once, instrument everywhere — spans and counters.



Log-level discipline matters. Use structured messages with meaningful attributes — not noisy string concatenation.



OTel gives you freedom to switch platforms. Your investment is portable — a config change, not a rewrite.



Baggage propagates context automatically — great for tenant ID, user ID, feature flags. Never PII.



Thank You, Beer City!

Questions & Discussion

Barret Blake · Architect, NimblePros



Resources

- opentelemetry.io/docs/what-is-opentelemetry/
- opentelemetry.io/docs/languages/dotnet/
- aspire.dev/dashboard/
- learn.microsoft.com — .NET observability with OTel
- github.com/barretb/MissionControlDemo



Contact Me

-  linkedin.com/in/barretblake
-  [bluesky: @barretblake.dev](https://bluesky.com/@barretblake.dev)
-  barretblake.dev
-  youtube.com/@barretcodes
-  blog.nimblepros.com